

Môn học

KIỂM THỬ PHẦN MỀM

Chương IV

CÁC KỸ THUẬT

THIẾT KẾ TEST

Nội dung

1. Xác định điều kiện Test và thiết kế Test cases
2. Phân loại các kỹ thuật thiết kế Test
3. Kỹ thuật kiểm thử hộp đen
4. Kỹ thuật kiểm thử hộp trắng
5. Kỹ thuật kiểm thử dựa trên kinh nghiệm
6. Chọn lựa kỹ thuật kiểm thử

IV.1 Xác định điều kiện Test và thiết kế Test cases

- ❑ 3 hoạt động quan trọng cần thực hiện trong quá trình kiểm thử
 - **Phân tích Test:** Xác định điều kiện Test
 - **Thiết kế Test:** Thiết kế Test cases
 - **Thực thi Test:** Xây dựng Test Procedures / Test Scripts (dùng cho test tự động)

Xác định kiểu kiện Test

- ❑ Dựa vào các **cơ sở test**
 - Tài liệu đặc tả yêu cầu
 - Code
 - Quy trình nghiệp vụ
 - Kiến thức của người dùng có kinh nghiệm
 - ...
- ❑ Từ cơ sở Test → Cái gì có thể được test?

Xác định kiểu kiện Test

□ Điều kiện test (test condition)

- Một mục hoặc sự kiện của hệ thống có thể được xác minh bởi 1 hoặc nhiều test case
- Ví dụ:
 - Các chức năng
 - Đặc trưng chất lượng
 -

Xác định kiểu kiện Test

- ❑ **Ví dụ 1:** Khi cần đo độ bao phủ nhánh
 - **Cơ sở Test:** code
 - **Các điều kiện Test:** các kết quả của nhánh (True or False)
- ❑ **Ví dụ 2:** Kiểm thử hệ thống tiếp thị và quản lý khách hàng
 - **Cơ sở Test:** đặc tả yêu cầu
 - **Các điều kiện Test:** liên quan chiến dịch quảng cáo như
 - Độ tuổi, giới tính của khách hàng
 - Zip code
 - Sở thích mua sắm,...

Xác định điều kiện Test

- ❑ Xu hướng xác định các điều kiện Test nhiều nhất có thể
- ❑ Kiểm thử triệt để (Exhaustive testing) không thực tế
 - Chỉ sử dụng tập nhỏ nhất các tests
 - Xác suất cao tìm ra hầu hết các lỗi
- ❑ Cần chọn lựa các tests một cách thông minh
 - Nhờ vào sử dụng các **kỹ thuật kiểm thử**

Thiết kế Test cases

□ **Formal Test case** là một đặc tả gồm các thành phần:

- Test case ID*
- Test summary/Description*
- Pre-condition
- Test steps*
- Inputs (test data)
- Expected result*
- Pass/Fail*
-

Thiết kế Test cases (TCs)

- ❑ TCs được thiết để xác minh sự thỏa mãn của test condition(test requirement)
- ❑ Mỗi test condition nên có tối thiểu 2 TCs
 - Positive test
 - Negative test
- ❑ Sau khi thiết kế xong, cần sắp xếp độ ưu tiên và tổ chức các TCs vào Test Procedure

Xây dựng Test Procedures / Test Scripts

❑ Test procedure

- Là các bước để chạy bằng tay một tập các test cases có liên quan

❑ Test script

- Là test procedure được viết bằng ngôn ngữ lập trình để chạy bằng các tool tự động

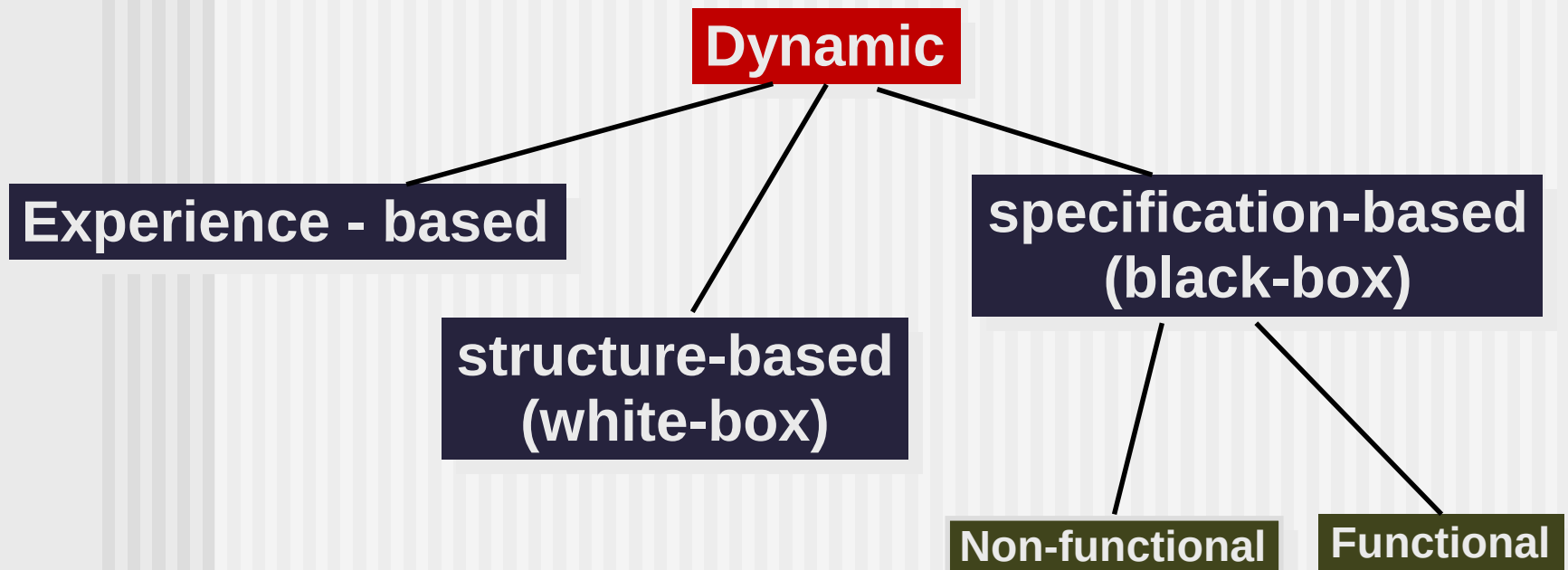
Kỹ thuật kiểm thử

- ❑ Là thủ tục chọn lựa hoặc thiết kế test tốt
- ❑ Dựa trên cấu trúc bên trong hoặc đặc tả của hệ thống PM
- ❑ Giúp tìm ra được nhiều lỗi
- ❑ Giúp đo nỗ lực test

Ưu điểm của các kỹ thuật

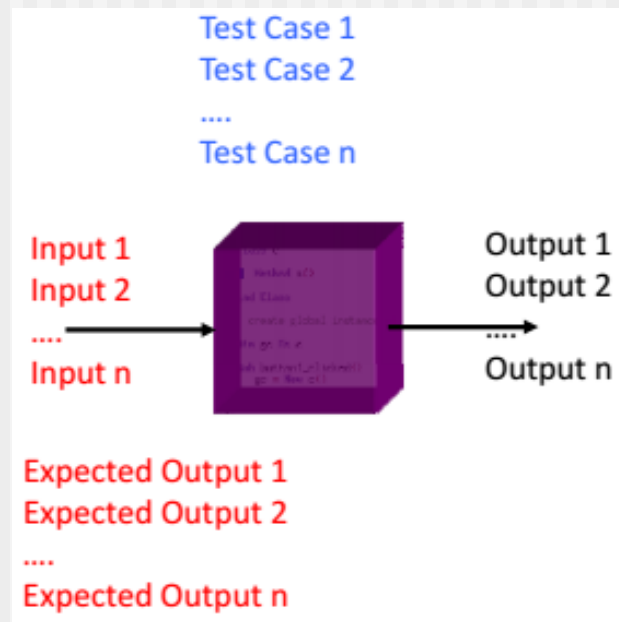
- ❑ Xác suất tìm ra lỗi của các tester là như nhau
- ❑ Kiểm thử hiệu quả hơn: Tìm ra được nhiều lỗi hơn với chi phí và nỗ lực ít
 - Tập trung vào các loại lỗi nhất định
 - Kiểm thử đúng hướng
 - Tránh trùng lặp

IV.2 Phân loại các kỹ thuật thiết kế Test



IV.3 Black-box testing

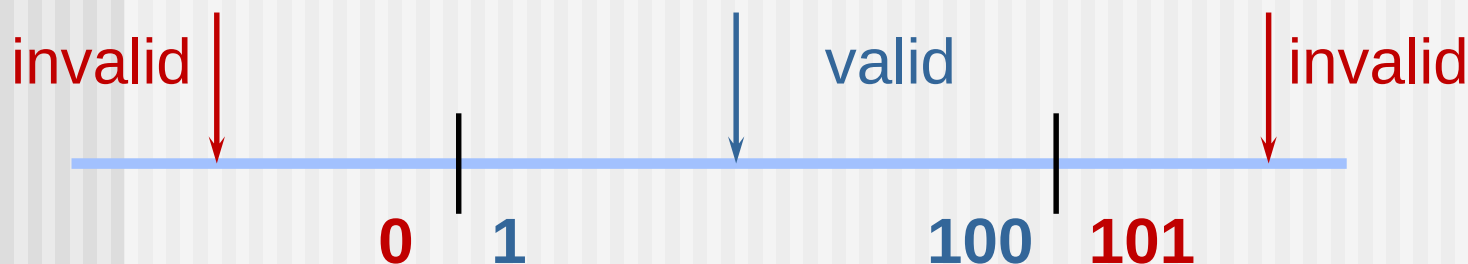
- ❑ Là phương pháp kiểm thử dựa trên hành vi hoặc chức năng của PM



Black-box testing

- ❑ Bao gồm các kĩ thuật:
 - Equivalence partitioning
 - Boundary value analysis
 - State transition testing
 - Decision table testing

Phân hoạch lớp tương đương



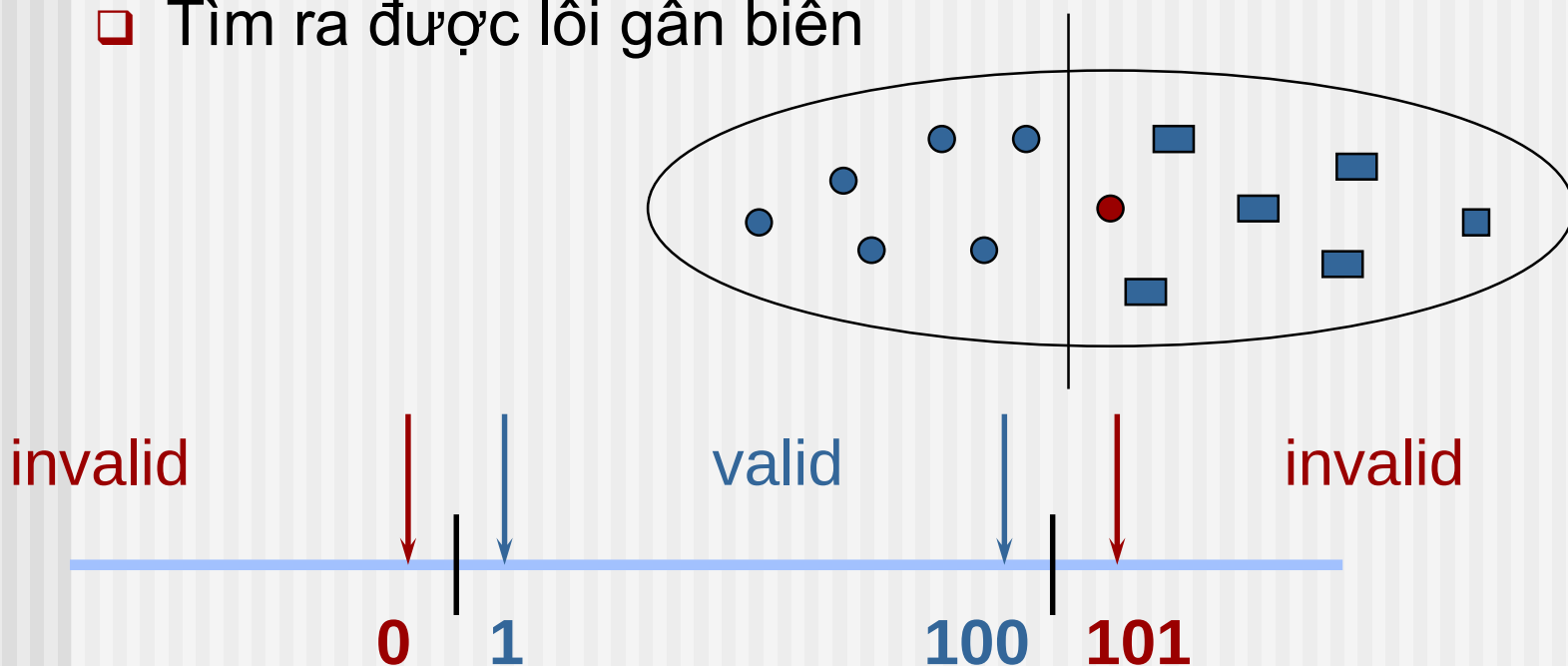
- ❑ Phân chia tập hợp các điều kiện test (dữ liệu đầu vào) thành những vùng/lớp tương đương nhau.
- ❑ Lớp tương đương(equivalence partitions)
 - 2 tests thuộc cùng lớp tương đương nếu kết quả mong đợi của nó giống nhau
 - → thực hiện nhiều tests thuộc cùng 1 lớp tương đương thì dẫn đến dư thừa

Phân hoạch lớp tương đương

STT	Trường hợp của điều kiện đầu vào	Lớp tương đương	
		Hợp lệ	Không hợp lệ
1	Là 1 khoảng các giá trị	1	2
	Ví dụ: [3,10]	$3 < x < 10$	$x < 3$ hoặc $x > 10$
2	Là 1 giá trị cụ thể	1	2
	Ví dụ: 6	$x = 6$	$x < 6$ hoặc $x > 6$
3	Là 1 tập hợp các giá trị	1	1
	Ví dụ: $A = \{1, 5, 4, 8, 6\}$	x	
4	Là 1 giá trị Boolean	1	1

Phân tích giá trị biên

- ❑ **Biên(boundary)** là điểm chuyển từ lớp tương đương này sang lớp tương đương khác
- ❑ Tìm ra được lỗi gần biên



Phân tích giá trị biên - Standard BVA

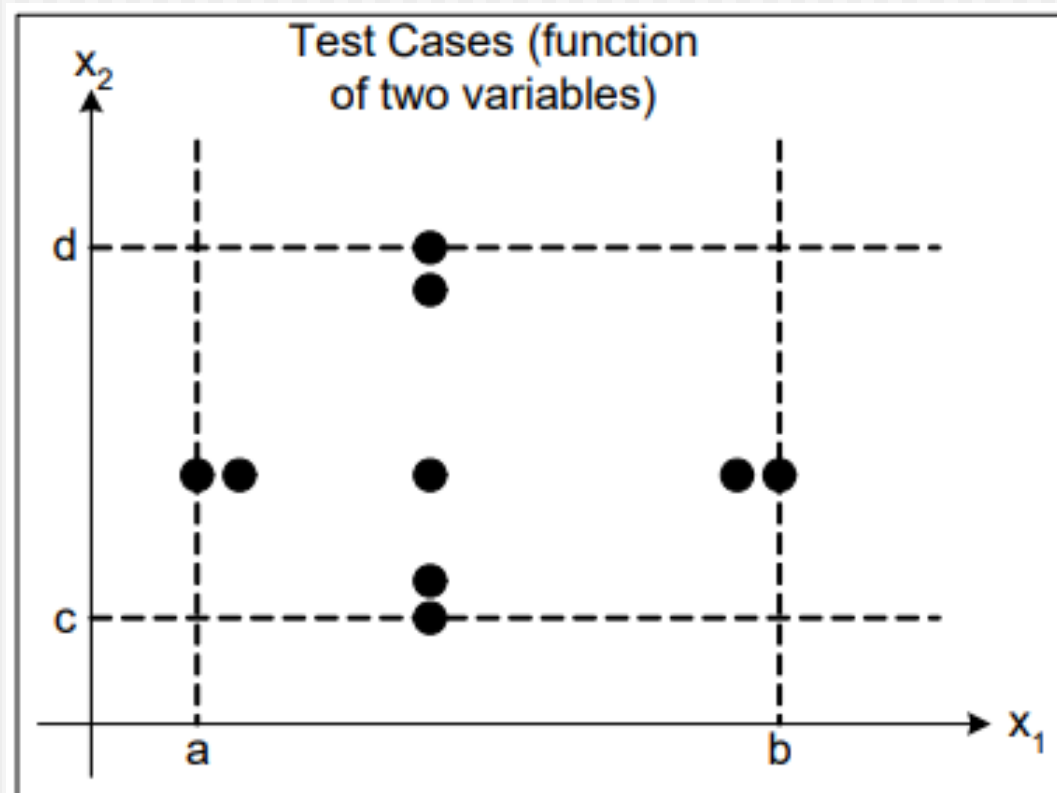
- Giả sử biến x có **miền giá trị $[min, max]$**

→ Các giá trị được chọn để kiểm tra

– Min	-	Minimal
– Min+	-	Just above Minimal
– Nom	-	Average
– Max-	-	Just below Maximum
– Max	-	Maximum

Phân tích giá trị biên - Standard BVA

□ Số TCs là $4n + 1$, với n là số lượng biến



VD: Bài toán kiểm tra tam giác

- Ràng buộc: $1 \leq a, b, c \leq 200$.
- Áp dụng Standard BVA (số test case $4 \cdot 3 + 1 = 13$)
 - min = 1
 - min+ = 2
 - nom = 100
 - max- = 199
 - max = 200

Boundary Value Analysis Test Cases				
Case	a	b	c	Expected Output
1	100	100	1	Isosceles
2	100	100	2	Isosceles
3	100	100	100	Equilateral
4	100	100	199	Isosceles
5	100	100	200	Not a Triangle
6	100	1	100	Isosceles
7	100	2	100	Isosceles
8	100	199	100	Isosceles
9	100	200	100	Not a Triangle
10	1	100	100	Isosceles
11	2	100	100	Isosceles
12	199	100	100	Isosceles
13	200	100	100	Not a Triangle

VD: Tìm ngày kế tiếp

- Bài toán tìm ngày kế tiếp với các ràng buộc:
 - $1 \leq \text{Day} \leq 31$.
 - $1 \leq \text{month} \leq 12$.
 - $1812 \leq \text{Year} \leq 2012$
- Áp dụng Standard BVA (số test case $4*3 + 1 = 13$)

Phân tích giá trị biên

Standard BVA

month
min = 1
min+ = 2
nom = 6
max- = 11
max = 12

day
min = 1
min+ = 2
nom = 15
max- = 30
max = 31

year
min = 1812
min+ = 1813
nom = 1912
max- = 2011
max = 2012

Boundary Value Analysis Test Cases

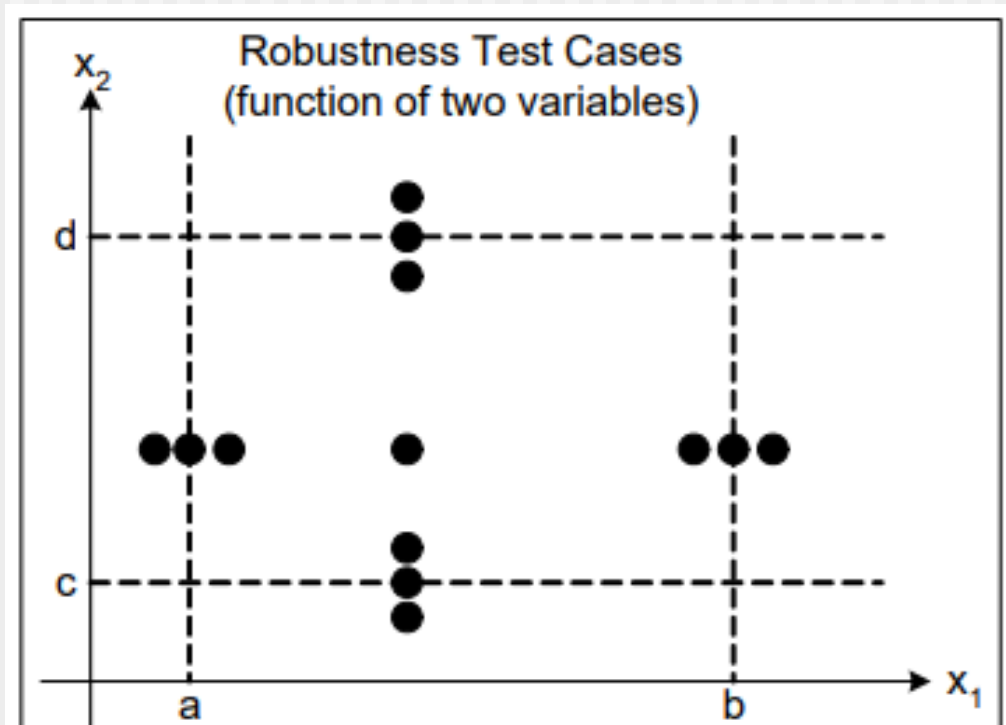
Case	month	day	year	Expected Output
1	6	15	1812	June 16, 1812
2	6	15	1813	June 16, 1813
3	6	15	1912	June 16, 1912
4	6	15	2011	June 16, 2011
5	6	15	2012	June 16, 2012
6	6	1	1912	June 2, 1912
7	6	2	1912	June 3, 1912
8	6	30	1912	July 1, 1912
9	6	31	1912	error
10	1	15	1912	January 16, 1912
11	2	15	1912	February 16, 1912
12	11	15	1912	November 16, 1912
13	12	15	1912	December 16, 1912

Phân tích giá trị biên - Robustness BVA

- Mở rộng của Standard BVA
- Kiểm thử cả hai trường hợp:
 - Input variable hợp lệ (clean test cases)
 - Kiểm thử tương tự như Standard BVA trên các giá trị (min, min+, average, max-, max)
 - Input variable không hợp lệ (dirty test cases)
 - Kiểm thử trên 2 giá trị: min-, max+ (nằm ngoài miền giá trị hợp lệ)

Phân tích giá trị biên - Robustness BVA

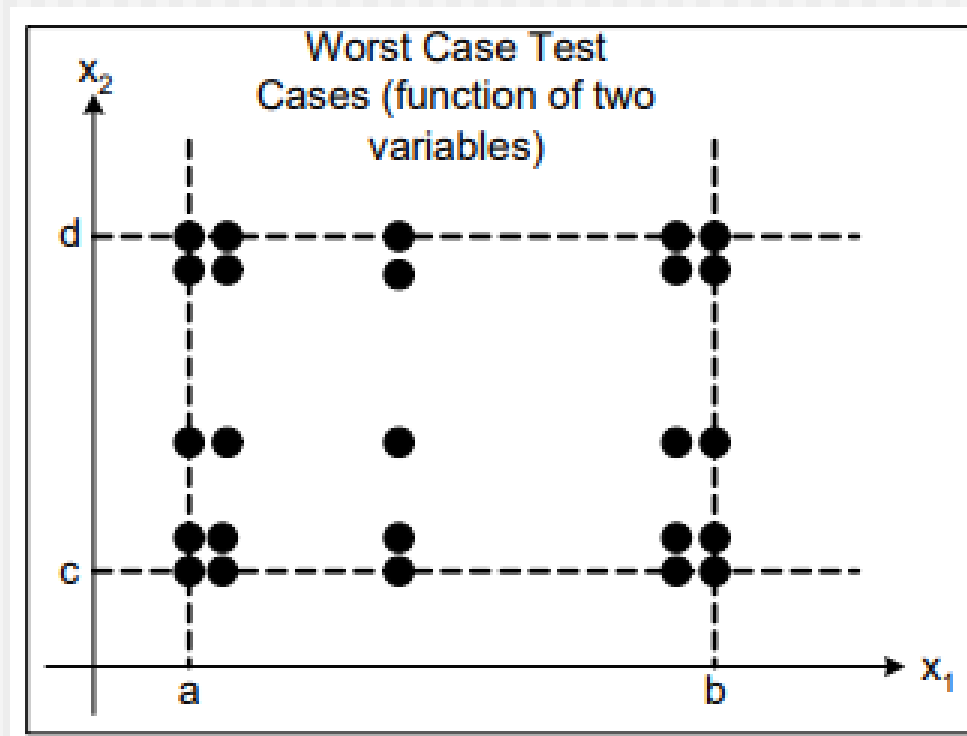
- ❑ Số TCs là $6n + 1$, với n là số lượng biến
- ❑ Tập trung vào việc kiểm thử trên giá trị không hợp lệ → ứng dụng phải xử lý ngoại lệ



Kiểm thử phần mềm

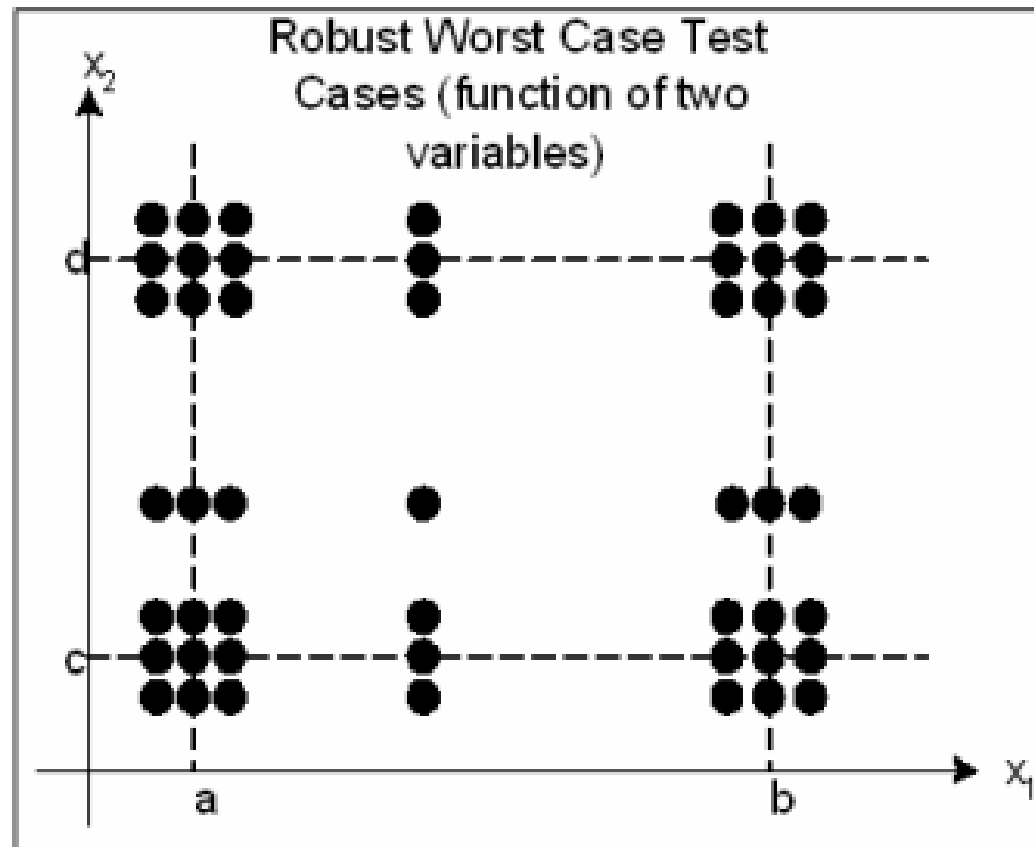
Worst-Case Testing

- ❑ Tại một thời điểm, BVA chỉ test giá trị biên của 1 biến → cần test giá trị biên của nhiều biến cùng lúc → **Số TCs là 5^n**



Robust Worst-Case Testing

- ❑ Số TCs là 7^n



Ví dụ: Loan application

Customer Name

2-64 chars.

Account number

6 digits, 1st non-zero

Loan amount requested

£500 to £9000

 Term of loan

1 to 30 years

 Monthly repayment

Minimum £10

Term:

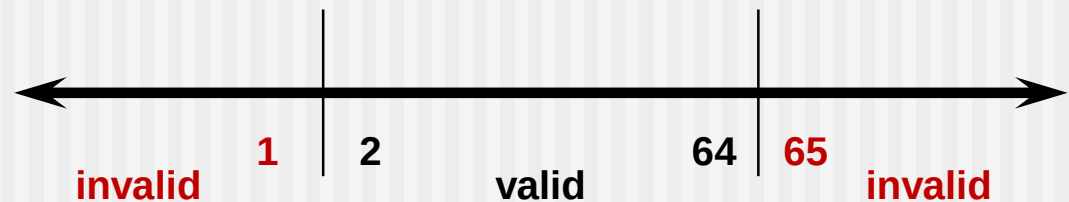
Repayment:

Interest rate:

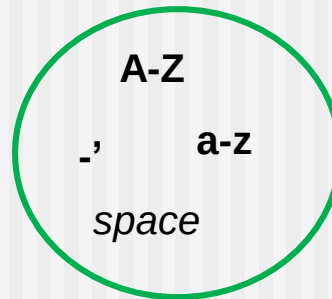
Total paid back:

Customer name

Số lượng kí tự:



Các kí tự hợp lệ:



Conditions	Valid Partitions	Invalid Partitions	Boundaries (Standard)
Customer name	2 to 64 chars valid chars	< 2 chars > 64 chars invalid chars	2 chars 3 chars 63 chars 64 chars 33 chars

Kiểm thử phần mềm

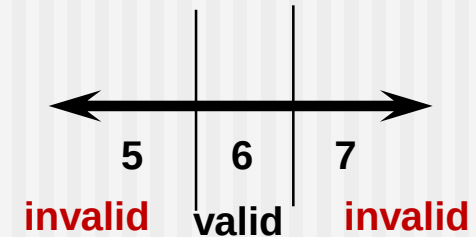
Account number

Chữ số đầu tiên:

valid: non-zero

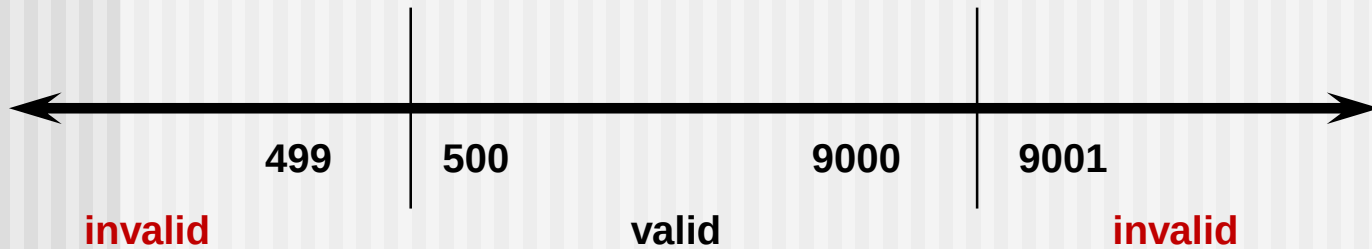
invalid: zero

Số lượng chữ số:



Conditions	Valid Partitions	Invalid Partitions	Boundaries (Standard)
Account number	6 digits 1 st non-zero digit	< 6 digits > 6 digits 1 st zero non-digit	100000 100001 500000 999998 999999

Loan amount



Conditions	Valid Partitions	Invalid Partitions	Boundaries (Standard)
Loan amount	500 – 9000 Numeric	< 500 >9000 non-numeric	500 5001 4750 8999 9000

Điều kiện test

Conditions	Valid Partitions	Tag	Invalid Partitions	Tag	Valid Boundaries	Tag
Customer name	2 - 64 chars valid chars	V1 V2	< 2 chars > 64 chars invalid char	X1 X2 X3	2 chars 3 chars 63 chars 64 chars 33 chars	B1 B2 B3 B4 B5
Account number	6 digits 1 st non-zero digit	V3 V4 V5	< 6 digits > 6 digits 1 st zero non-digit	X4 X5 X6 X7	100000 100001 500000 999998 999999	B6 B7 B8 B9 B10
Loan amount	500 – 9000 Numeric	V6 V7	< 500 >9000 non-numeric	X8 X9 X10	500 5001 4750 8999 9000	B11 B12 B13 B14 B15

Thiết kế test cases

Test Case	Input	Expected Outcome		New Tags Covered
1	Name: John Smith Acc no: 123456 Loan: 2500 Term: 3 years	Term:	3 years	V1, V2, V3, V4, V5
		Repayment:	79.86	
		Interest rate:	10%	
		Total paid:	2874.96	
2	Name: AB Acc no: 100000 Loan: 500 Term: 1 year	Term:	1 year	B1, B6, B11,
		Repayment:	44.80	
		Interest rate:	7.5%	
		Total paid:	537.60	

Bài tập 1

Một tài khoản tiết kiệm trong một ngân hàng nhận được mức lãi suất khác nhau tùy thuộc vào số dư trong tài khoản:

- Số dư 1\$ - 100\$ → lãi suất 3%
- Số dư lớn hơn 100\$ - nhỏ hơn 1000\$ → lãi suất 5%
- Số dư từ 1000\$ trở lên → lãi suất 7%

Sử dụng kỹ thuật phân hoạch lớp tương đương và phân tích giá trị biên để thiết kế các test cases tối thiểu cho yêu cầu trên.

Bài tập 2

Xây dựng chương trình sau:

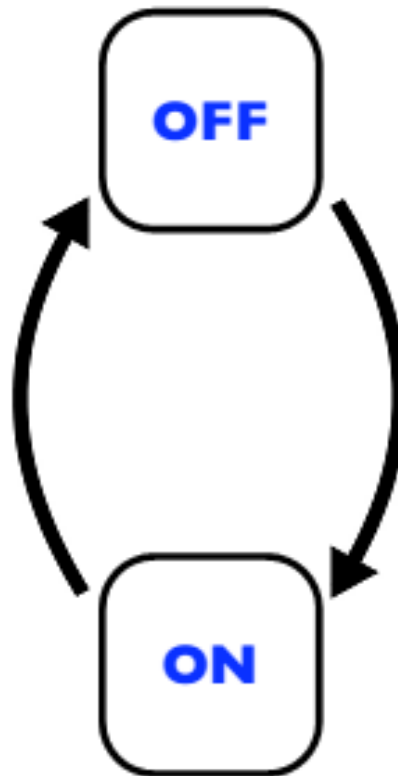
- ❑ Nhập vào 1 tháng
- ❑ Xuất Quý tương ứng của tháng

Hãy thiết kế các test case để test chương trình trên với kỹ thuật phân hoạch lớp tương đương (EP) và phân tích giá trị biên (BVA).

Chuyển đổi trạng thái (State Transition Testing)

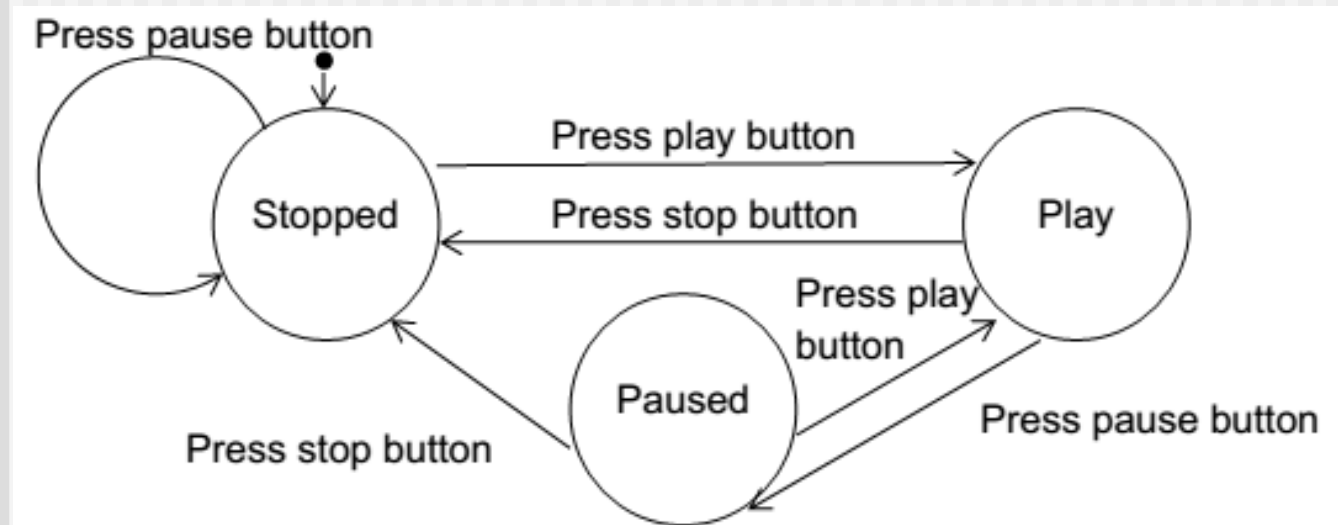
- Phân tích mối quan hệ giữa các trạng thái, sự kiện, hành động(action) gây ra sự chuyển đổi từ trạng thái này sang trạng thái khác
- **Các bước thực hiện**
 - Xác định tất cả các trạng thái
 - Với mỗi test, xác định
 - Start state
 - Input
 - Expected Output/End state

Ví dụ 1:



TEST 1	TEST 2
start state: off	start state: on
input: switch on	input: switch off
output: light on	output: light off
finish state: on	finish state: off

Ví dụ 2: Media Player

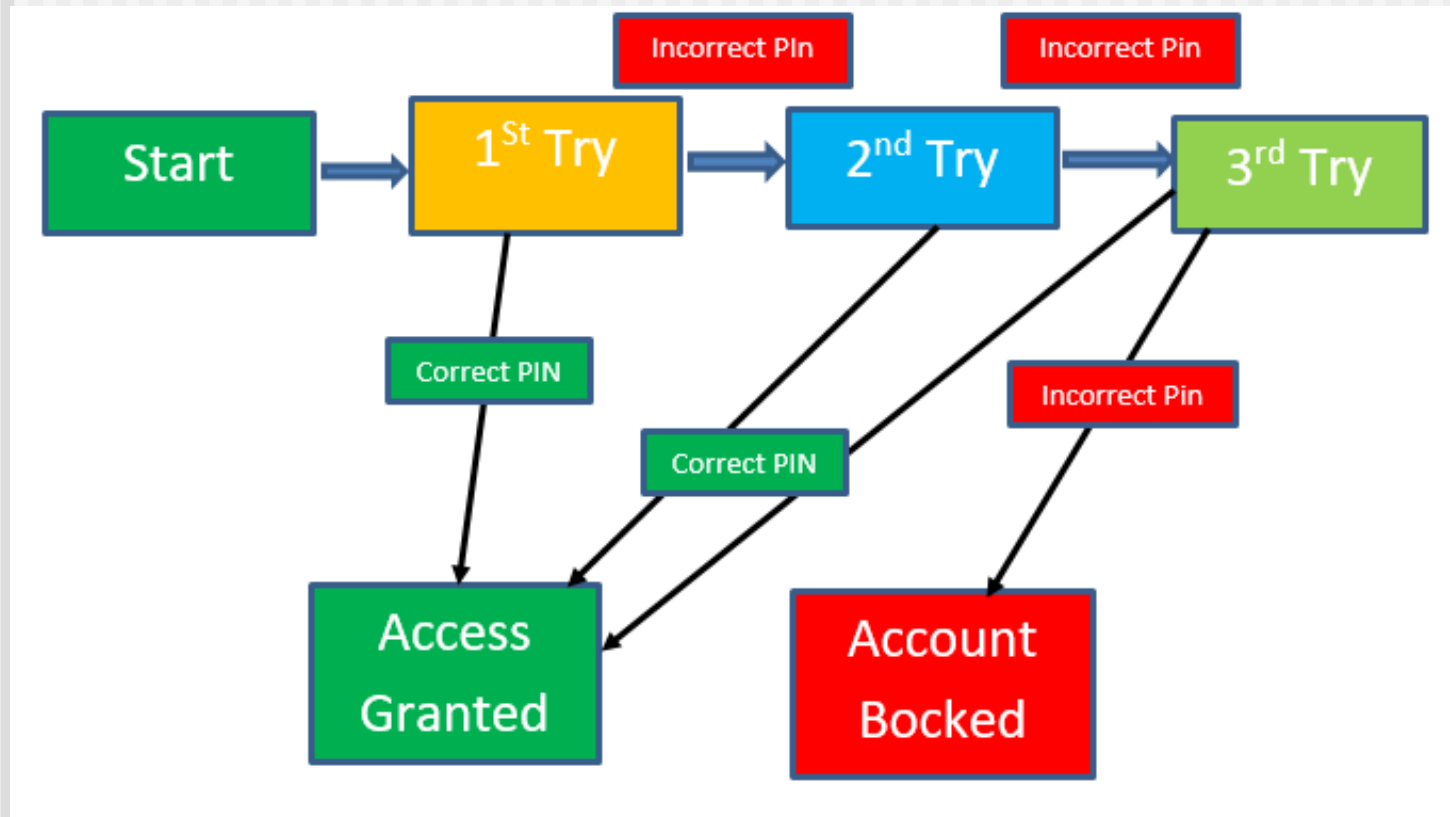


Ví dụ 2: Media Player

Test Case No.	Start State	Event/Input	End State/ Exp Output
1	Stopped	Press play button	Play
2	Stopped	Press pause button	Stopped
3	Play	Press stop button	Stopped
4	Play	Press pause button	Paused
5	Paused	Press stop button	Stopped
6	Paused	Press play button	Play

Bài tập

Xây dựng các test case ứng với sơ đồ chuyển trạng thái sau:



Kiểm thử phần mềm

Bảng quyết định (Decision Table)

□ Gồm 3 phần:

■ Conditions

- Các điều kiện đầu vào để ra quyết định

■ Actions

- Kết quả của sự kết hợp các điều kiện đầu vào

■ Rules

- Luật kết hợp các điều kiện

Bảng quyết định (Decision Table)

❑ Các bước thực hiện tạo bảng:

1. Liệt kê tất cả các **conditions/inputs** và xác định giá trị của nó
2. Liệt kê tất cả các **actions**
3. Tính số **rules** lớn nhất
4. Xác định các **rules** từ sự tổ hợp giá trị của các **conditions/inputs**
5. Đánh dấu “X” vào mỗi **actions** tương ứng với mỗi **rule**
6. Rút gọn bảng (nếu có thể)

Ví dụ 1:

- Một ngân hàng sử dụng những luật sau để phân loại tài khoản mới của người gửi tiền:
 - Nếu tuổi ≥ 21 và số tiền gửi là ≥ 100 nghìn, thì loại tài khoản là A
 - Nếu tuổi < 21 và số tiền gửi là ≥ 100 nghìn, thì loại tài khoản là B
 - Nếu tuổi ≥ 21 và số tiền gửi là < 100 nghìn, thì loại tài khoản là C
 - Nếu tuổi < 21 và số tiền gửi là < 100 nghìn, thì không thể mở tài khoản

Ví dụ 1:

	Condition/Action	R1	R2	R3	R4
C1	Tuổi ≥ 21	Y	Y	N	N
C2	Tiền ≥ 100 nghìn	Y	N	Y	N
A1	A	X	-	-	-
A2	B	-		X	-
A3	C	-	X	-	-
A4	Ko thể mở TK	-	-	-	X

Ví dụ 2:

- ❑ Để tính thuế thu nhập, người ta có mô tả sau:

- *Người vô gia cư nộp 4% thuế thu nhập*
- *Người có nhà ở nộp thuế theo bảng sau:*

Tổng thu nhập	Thuế
$\leq 5.000.000$ đồng	4%
$> 5.000.000$ đồng	6%

Ví dụ 2:

	Condition/Action	R1	R2	R3	R4
C1	Người có nhà ở	Y	Y	N	N
C2	Tổng thu nhập ≤ 5000000	Y	N	Y	N
A1	Nộp thuế 4%	X	-	X	X
A2	Nộp thuế 6%	-	X	-	-

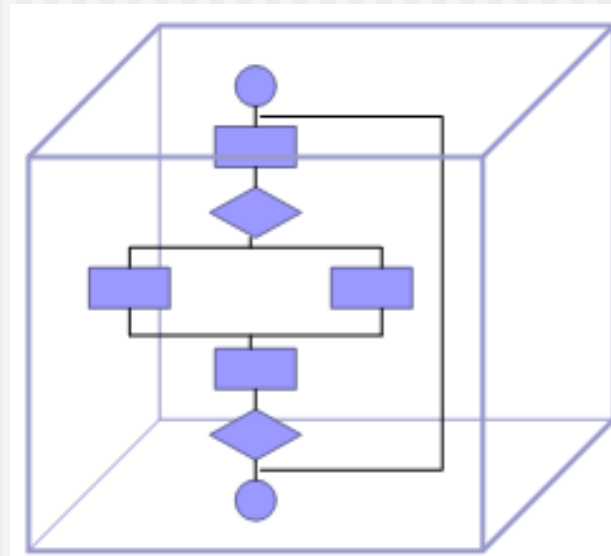
	Condition/Action	R1	R2	R3
C1	Người có nhà ở	Y	Y	N
C2	Tổng thu nhập ≤ 5000000	Y	N	-
A1	Nộp thuế 4%	X	-	X
A2	Nộp thuế 6%	-	X	-

Bài tập

1. Vẽ bảng quyết định để chọn số lớn nhất trong 3 số khác nhau A, B, C
2. Một công ty bán hàng cần tuyển dụng một số nhân viên bán hàng thỏa mãn các yêu cầu:
 - Người chưa kết hôn ở độ tuổi từ 18 đến 30.
 - Nếu là nam thì phải cao từ 1.6m trở lên và cân nặng không quá 70kg, không bị hói. Nếu là nữ thì phải cao từ 1.55m trở lên và cân nặng không quá 55kg, tóc dài ngang vai

IV.4 White-box testing

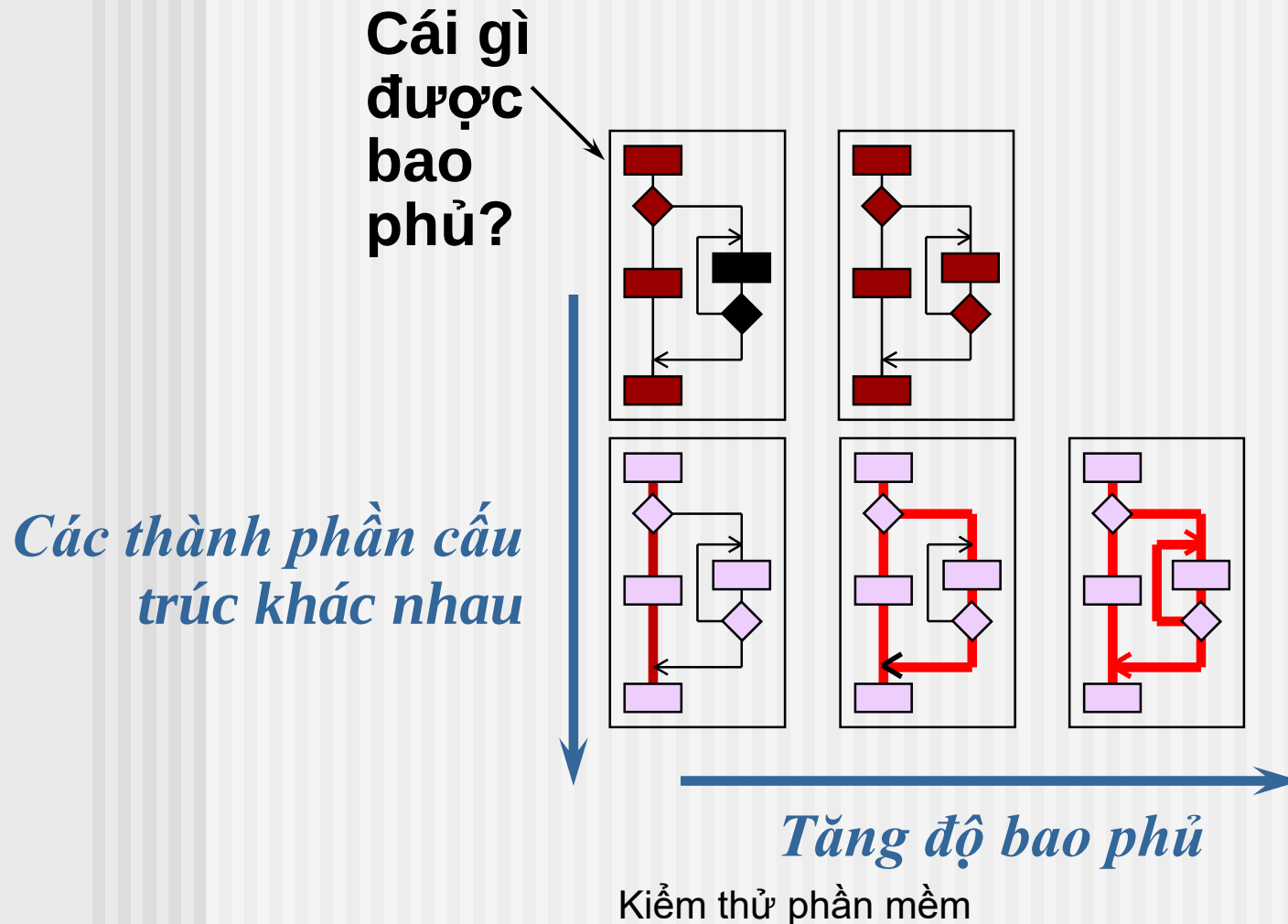
- ❑ Là phương pháp kiểm thử dựa trên cấu trúc logic của PM



White-box testing

- ❑ Control Flow Testing
 - Statement testing
 - Branch / Decision testing
 - Branch condition testing
 - Branch condition combination testing
- ❑ Data flow testing

Kĩ thuật độ bao phủ cấu trúc (Coverage techniques)



Kĩ thuật độ bao phủ cấu trúc (Coverage techniques)

- ❑ Đánh giá độ bao phủ của các thành phần cấu trúc
- ❑ Thiết kế các test bổ sung → có thể đạt được mức bao phủ 100%

*Độ bao phủ 100%
không có nghĩa là
100% được test*

Kiểm thử luồng điều khiển (Control Flow Testing)

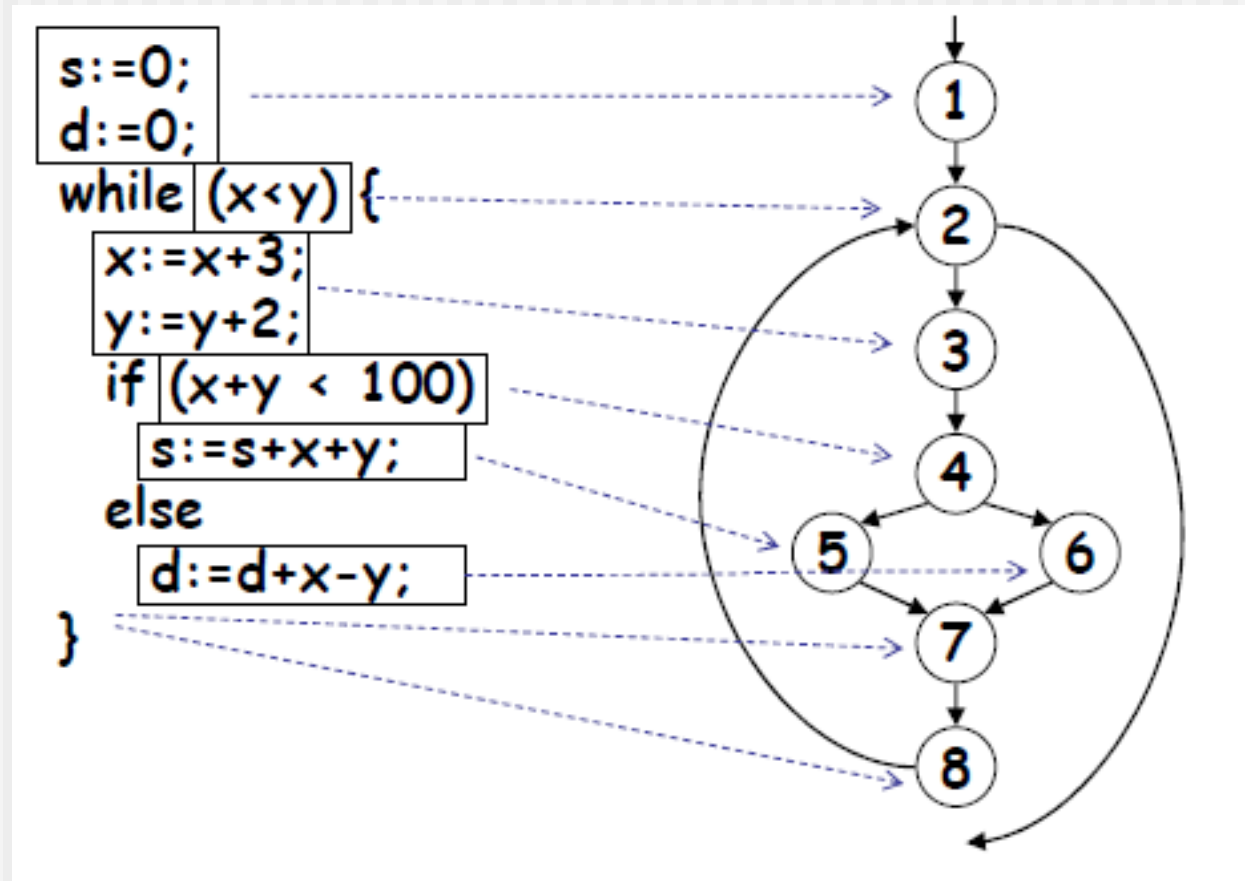
❑ Bước 1:

- Xây dựng đồ thị luồng điều khiển từ source code
- Đồ thị bao gồm nút, cạnh

❑ Bước 2:

- Thiết kế các Test cases để bao phủ hết các thành phần của đồ thị

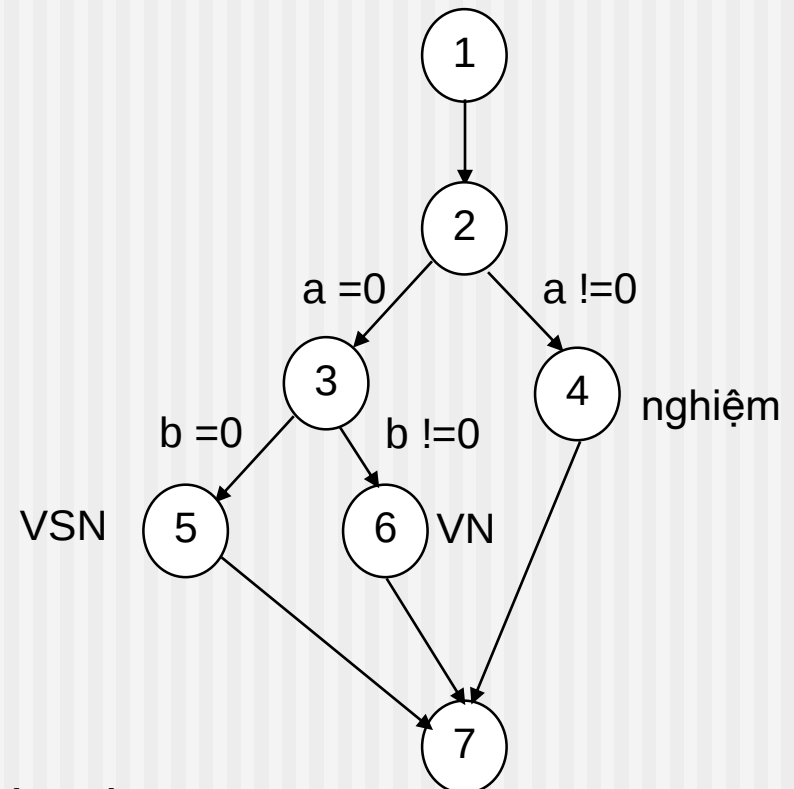
Ví dụ 1:



Ví dụ 2:

Đồ thị luồng điều khiển của đoạn chương trình giải phương trình bậc nhất

1. if(a == 0)
2. if(b == 0)
3. cout<<"Vo so nghiem";
4. else
5. cout<<"Vo nghiem";
6. else
7. cout<<"x = "<<-b/a;



Độ phức tạp “Cyclomatic”

- $V(G) = E - N + 2$
- $V(G) = P + 1$
- $V(G) = R$

Trong đó:

- E: số cạnh
- N: số nút
- P: số nút điều kiện
- R: số miền

Vẽ đồ thị luồng điều khiển

```
1. void prime(int n){ //1
2.     cout<<"Cac so nguyen to nho hon n la:"<<endl; //2
3.     for(int i = 2 //3; i < n//4;i++//5){
4.         int flag = 1;          //6
5.         for(int j = 2 //7; j <= sqrt(i) //8;j++ //9)
6.             if(i%j==0) //10
7.                 { flag = 0; break;} //11
8.         if(flag == 1){          //12
9.             cout<<i<<"\t"; //13
10.        }
11.    }//14
12. }
```

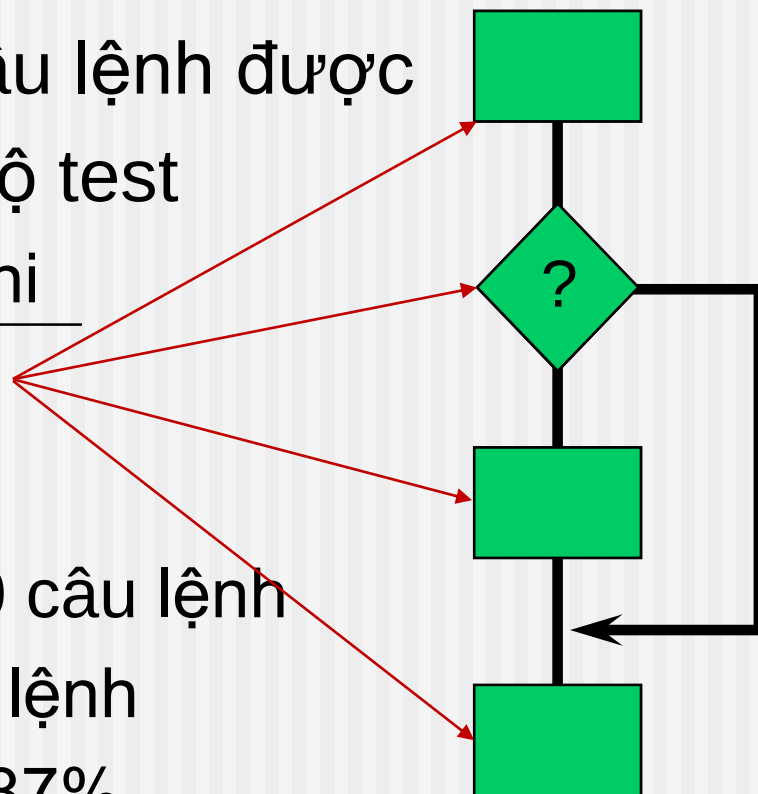

Bao phủ câu lệnh (Statement coverage)

- ❑ Tỷ lệ phần trăm các câu lệnh được thực thi bởi bộ test

$$= \frac{\text{Số câu lệnh thực thi}}{\text{Tổng số câu lệnh}}$$

- ❑ Ví dụ:

- ❑ Chương trình có 100 câu lệnh
- ❑ tests thực thi 87 câu lệnh
- ❑ Bao phủ câu lệnh = 87%



Ví dụ 1:

1	<code>cin>>a;</code>
2	<code>if (a > 6)</code>
3	<code> b = a;</code>
4	<code>cout<<b;</code>

Số câu lệnh

Test case	Input	Expected output
1	7	7

Với 1 test case này thì tất cả 4 câu lệnh đều được thực thi
→ đạt độ bao phủ dòng lệnh 100%

Kiểm thử phần mềm

Ví dụ 2:

```
Prints (int a, int b) {  
    int result = a+ b;  
    If (result> 0)  
        Print ("Positive", result)  
    Else  
        Print ("Negative", result)  
}
```

Ví dụ 2:

- ❑ Với $A = 2$, $B = 5 \rightarrow$ phủ 5/7 câu lệnh (71%)

```
1 Print (int a, int b) {  
2   int result = a + b;  
3   If (result > 0)  
4       Print ("Positive", result)  
5   Else  
6       Print ("Negative", result)  
7 }
```

Ví dụ 2:

- ❑ Với $A = 3$, $B = -5 \rightarrow$ phủ 6/7 câu lệnh (85%)

```
1 Prints (int a, int b) {  
2   int result = a+ b;  
3   If (result> 0)  
4       Print ("Positive", result)  
5   Else  
6       Print ("Negative", result)  
7 }
```

Ví dụ 2:

Tối thiểu 2 test-case để phủ 100% câu lệnh

```
Prints (int a, int b) {  
    int result = a+ b;  
    If (result> 0)  
        Print ("Positive", result)  
    Else  
        Print ("Negative", result)  
}
```

Bài tập 1

Đoạn chương trình giải phương trình bậc nhất $ax+b=0$

```
1.  {  
2.  if(a == 0)  
3.      if(b == 0)  
4.          cout<<"Vo so nghiem";  
5.      else  
6.          cout<<"Vo nghiem";  
7.  else  
8.      cout<<"x ="<<-b/a;  
9.  }
```

1. Có bao nhiêu câu lệnh?
 2. Test case ($a=0, b=8$) bao phủ bao nhiêu % câu lệnh ?
 3. Cần tối thiểu bao nhiêu test case để bao phủ 100% các câu lệnh ?
- Kiểm thử phần mềm

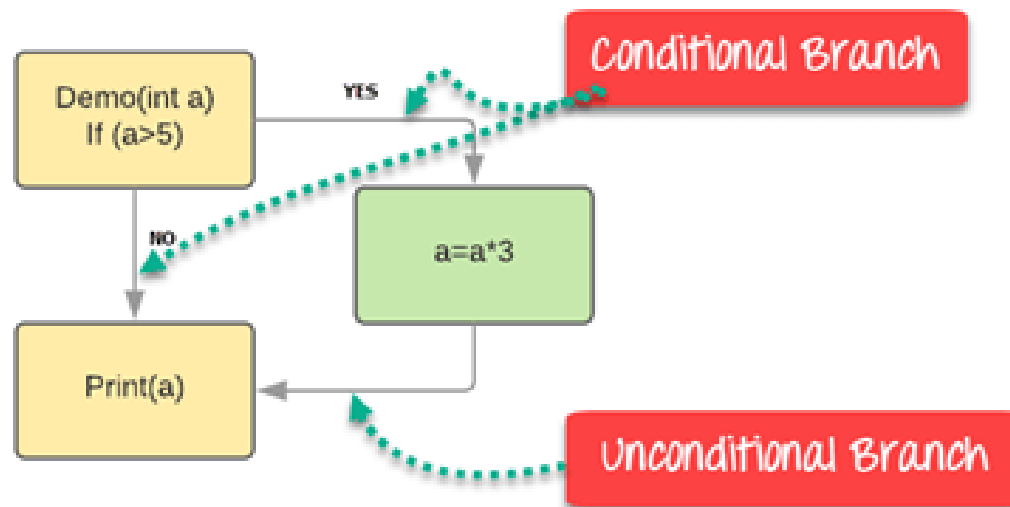
Bài tập 2

```
1. void func(int A, int B, int X )
2. {
3.     if (A > 1 && B == 0 )
4.         X = X / A;
5.     if ( A == 2 || X > 1)
6.         X = X + 1;
7. }
```

-
1. Có bao nhiêu câu lệnh ?
 2. Test case (A=2,B=1,X=3) bao phủ bao nhiêu % câu lệnh ?
 3. Test case (A=3,B=0,X=0) bao phủ bao nhiêu % câu lệnh ?
 4. Tối thiểu bao nhiêu test case để bao phủ 100% các câu lệnh ?

Bao phủ nhánh (Decision/Branch coverage)

```
Demo(int a)
{
    If (a > 5)
        a = a * 3
    Print (a)
}
```



Kiểm thử phân mềm

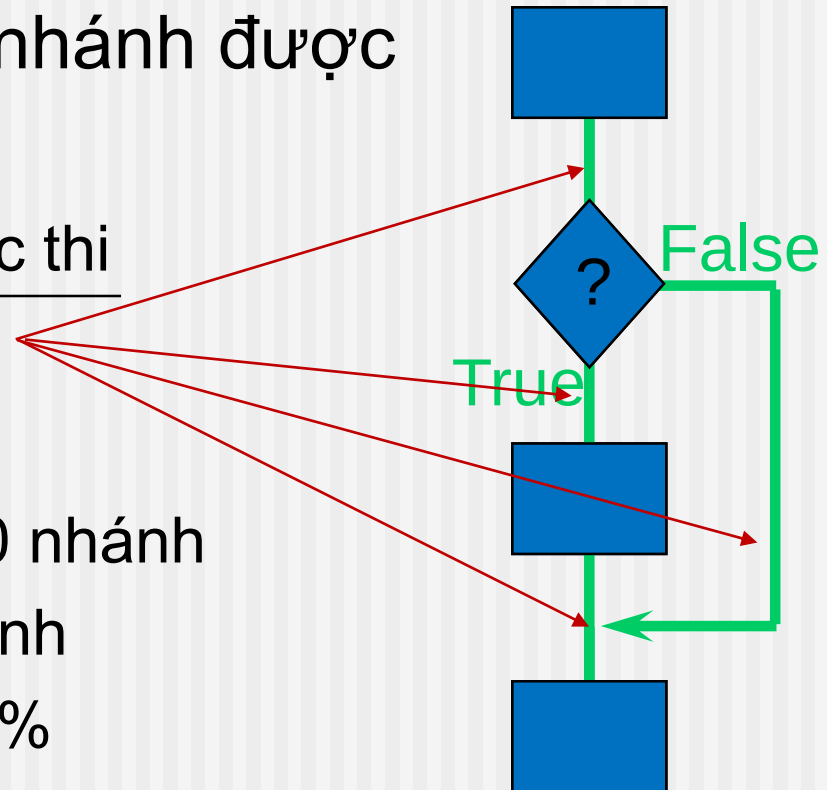
Bao phủ nhánh (Decision/Branch coverage)

- ❑ Tỷ lệ phần trăm các nhánh được thực thi bởi bộ test

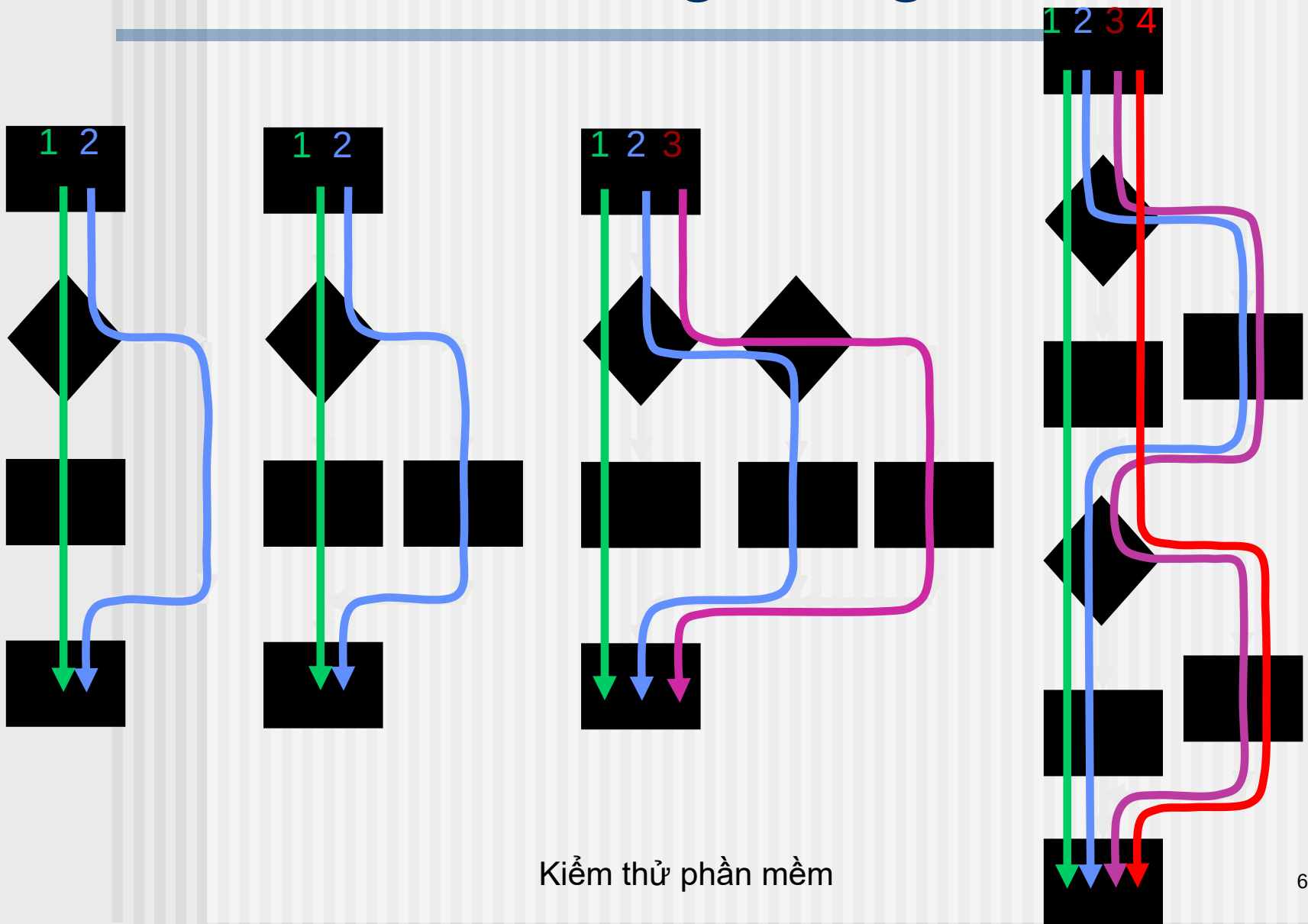
$$= \frac{\text{Số nhánh được thực thi}}{\text{Tổng số nhánh}}$$

- ❑ Ví dụ:

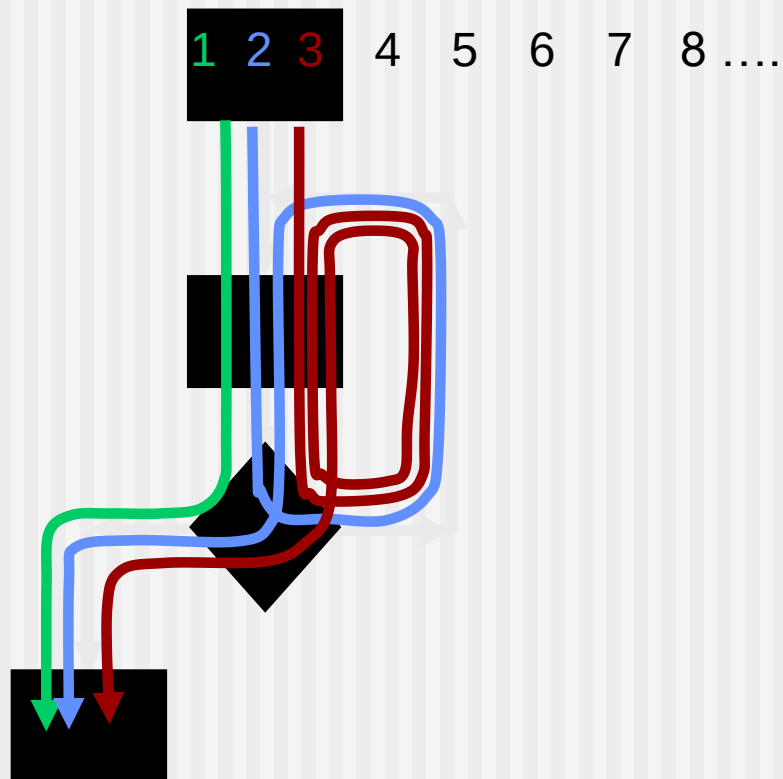
- Chương trình có 120 nhánh
- tests thực thi 60 nhánh
- Bao phủ nhánh = 50%



Tất cả các đường trong code



Các đường trong vòng lặp



Ví dụ 1:

Đoạn chương trình giải phương trình bậc nhất $ax+b=0$

```
1.  {  
2.  if(a == 0)  
3.      if(b == 0)  
4.          cout<<"Vo so nghiem";  
5.      else  
6.          cout<<"Vo nghiem";  
7.  else  
8.      cout<<"x =",-b/a;  
9.  }
```

1. Có bao nhiêu nhánh ?
2. Test case $(a=0,b=8), (a=0,b=0)$ bao phủ bao nhiêu % nhánh ?
3. Tối thiểu bao nhiêu test case để bao phủ 100% các nhánh?

Ví dụ 2:

```
void func(int A, int B, int X )  
1.  {  
2.      if ( A > 1 && B == 0 )  
3.          X = X / A;  
4.      if ( A == 2 || X > 1)  
5.          X = X + 1;  
6.  }
```

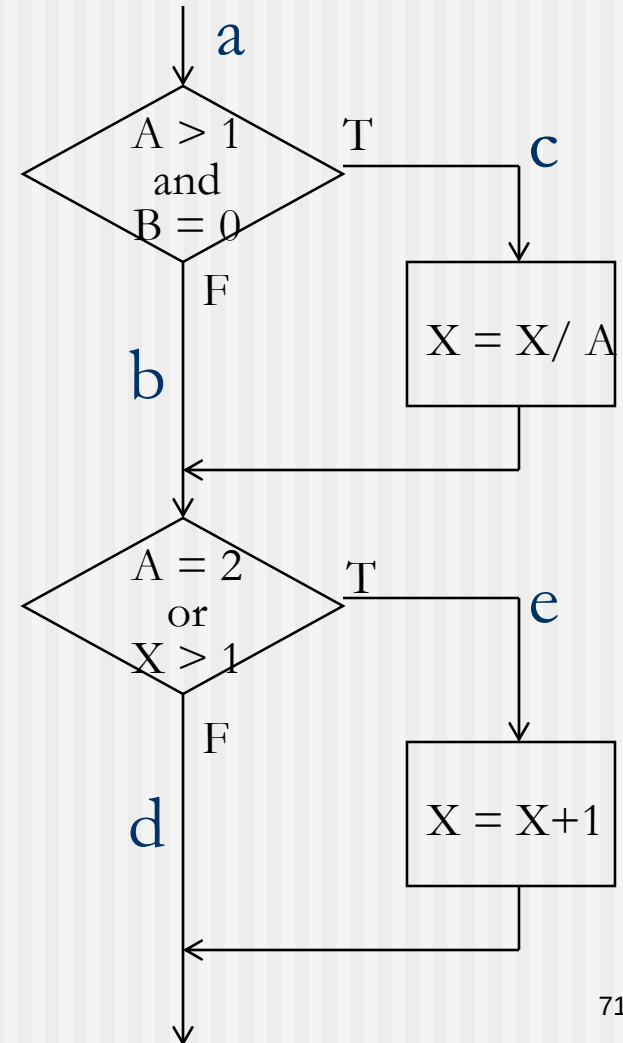
-
1. Có bao nhiêu nhánh?
 2. Test case (A=2,B=0,X=3) bao phủ bao nhiêu % nhánh ?
 3. Tối thiểu bao nhiêu test case để bao phủ 100% các nhánh ?

Bao phủ nhánh (điểm yếu ?)

```
void func(int A, int B, int X)
{
    if ( A > 1 && B == 0 )
        X = X / A;
    if ( A == 2 || X > 1 )
        X = X + 1;
}
```

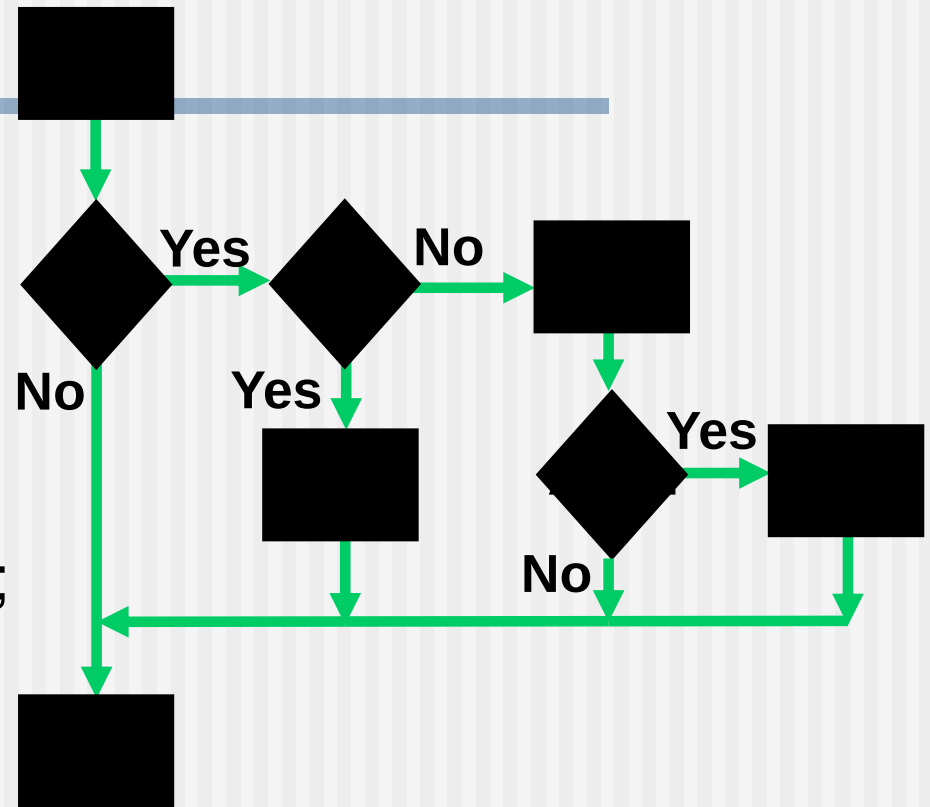
Test cases bao phủ nhánh

- 1) A = 3, B = 0, X = 3 (acd)
- 2) A = 2, B = 1, X = 1 (abe)



Ví dụ

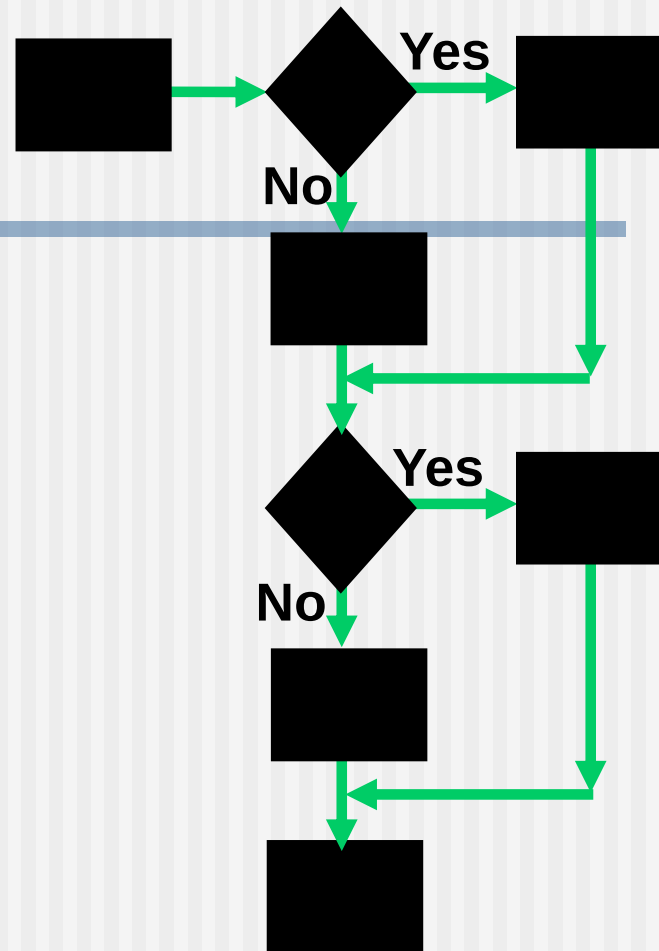
```
1. cin>>A;
2. cin>> B;
3. if (A > 0)
4.     if (B == 0)
5.         cout<< "No values";
6.     else
7.     {
8.         cout<<"B"<<endl;
9.         if (A > 21)
10.            cout<<"A";
11.     }
```



- Độ phức tạp Cyclomatic : 4
- Số tests nhỏ nhất:
 - Statement coverage: 2
 - Branch coverage: 4

Ví dụ:

1. Read A
2. Read B
3. IF A < 0 THEN
4. Print "A negative"
5. ELSE
6. Print "A positive"
7. ENDIF
8. IF B < 0 THEN
9. Print "B negative"
10. ELSE
11. Print "B positive"
12. ENDIF



- Độ phức tạp Cyclomatic : 3
- Số tests nhỏ nhất :
 - Statement coverage: 2
 - Branch coverage: 2

Bao phủ điều kiện (Condition coverage)

□ Tỷ lệ phần trăm các điều kiện được thực thi bởi bộ test

$$= \frac{\text{Số điều kiện được thực thi}}{\text{Tổng số điều kiện}}$$

Ví dụ: Bao phủ điều kiện

- Cần thiết kế các test cases bao phủ hết các điều kiện sau:

$A > 1$, $A \leq 1$, $B = 0$, $B \neq 0$

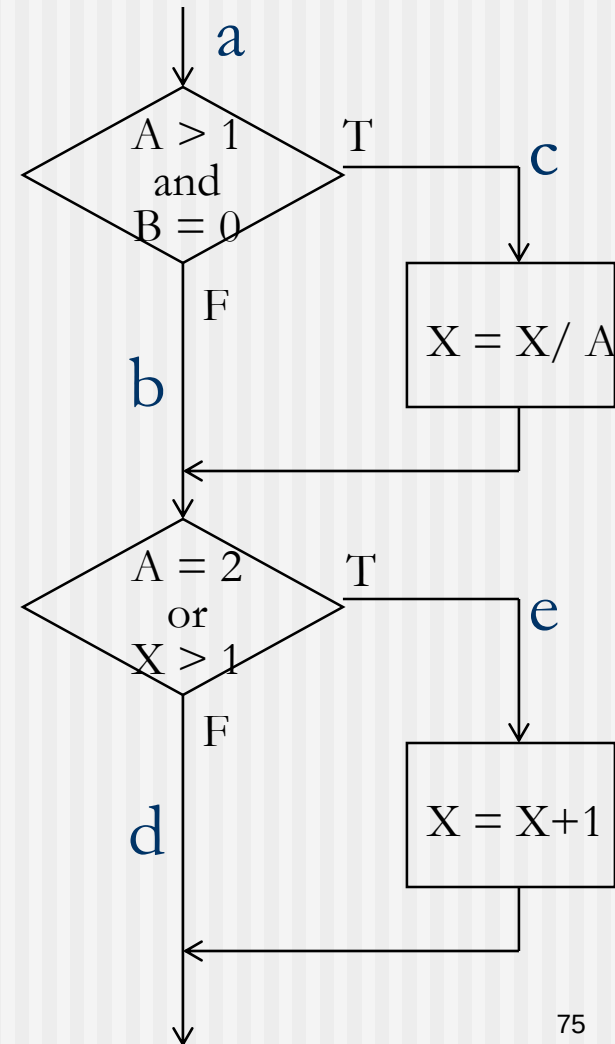
$A = 2$, $A \neq 2$, $X > 1$, $X \leq 1$

- Test cases:

1) $A = 1$, $B = 0$, $X = 3$ (abe)

2) $A = 2$, $B = 1$, $X = 1$ (abe)

- Không thỏa mãn bao phủ nhánh



Bài tập 1

```
1. float A, B, C;
2. printf("Enter three values\n");
3. scanf("%f%f%f", &A, &B, &C);
4. printf("\n largest value is: ");
5. if( A>B)
6. {
7.     if (A>C) printf("%f\n",A);
8.     else printf("%f\n",C);
9. }
10. else
11. {
12.     if (C>B) printf("%f\n",C);
13.     else printf("%f\n",B);
14. }
```

Bài tập 2

```
1. void PTA(int a[], int n)
2. {
3.     int i = 0;
4.     while ((i < n) && (a[i] >= 0))
5.         i++;
6.     if (i < n)
7.         printf("Phan tu am a[%d] = %d", i, a[i]);
8.     else
9.         printf("Khong co phan tu am.");
10. }
```

Bài tập 3

```
1.  const int SCALENE = 1;
2.  const int ISOSCELES = 2;
3.  const int EQUILATERAL = 3;
4.  const int ERROR = 4;
5.  int TriangleType(int x, int y, int z){
6.      if (x<=0 || y<=0 || z<=0)
7.          return ERROR;
8.      if ((x + y <= z) || (x + z <= y) || (z + y <= x))
9.          return ERROR;
10.     if (x == y && y == z)
11.         return EQUILATERAL;
12.     if (x == y || y == z || z == x)
13.         return ISOSCELES;
14.     return SCALENE;
15. }
```

Bao phủ nhánh – điều kiện

□ Bao phủ nhánh - điều kiện

- Viết các TCs sao cho mỗi điều kiện (đơn) trong mỗi nhánh nhận được 2 giá trị (T và F) ít nhất 1 lần và mỗi nhánh được thực hiện ít nhất 1 lần

□ Bao phủ tổ hợp các điều kiện

- Viết các TCs để thực thi được tất cả các tổ hợp giá trị (T và F) của các điều kiện (đơn) trong 1 nhánh

Ví dụ: Bao phủ nhánh - điều kiện

❑ Các TCs cần bao phủ tất các điều kiện

$A > 1$, $A \leq 1$, $B = 0$, $B \neq 0$

$A = 2$, $A \neq 2$, $X > 1$, $X \leq 1$

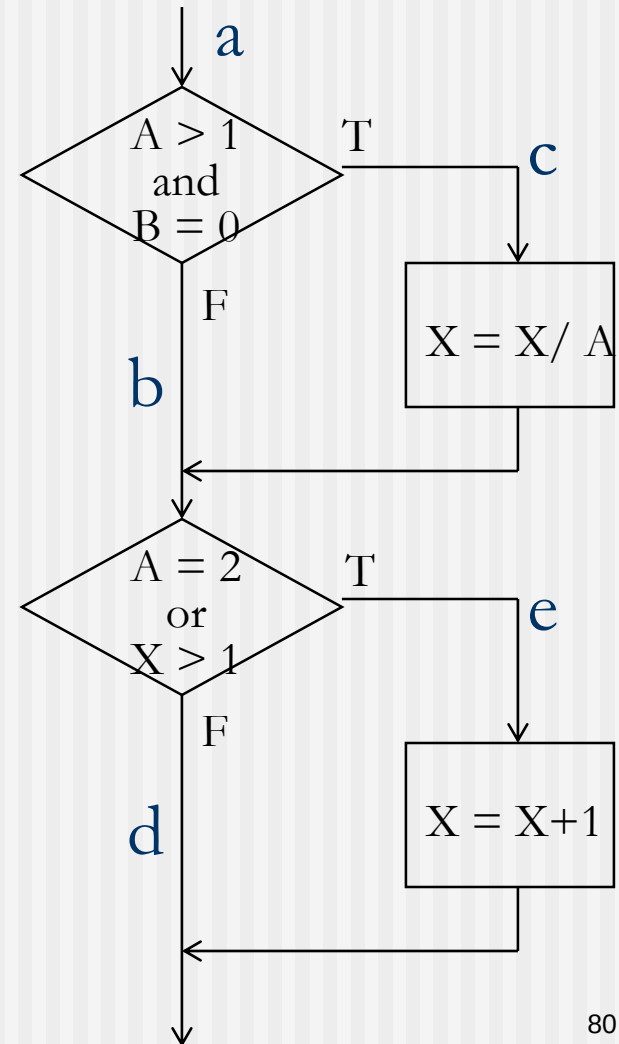
và $(A > 1 \text{ and } B = 0)$ T, F

$(A = 2 \text{ or } X > 1)$ T, F

❑ Test cases:

1) $A = 2$, $B = 0$, $X = 4$ (ace)

2) $A = 1$, $B = 1$, $X = 1$ (abd)



Ví dụ: Bao phủ tổ hợp điều kiện

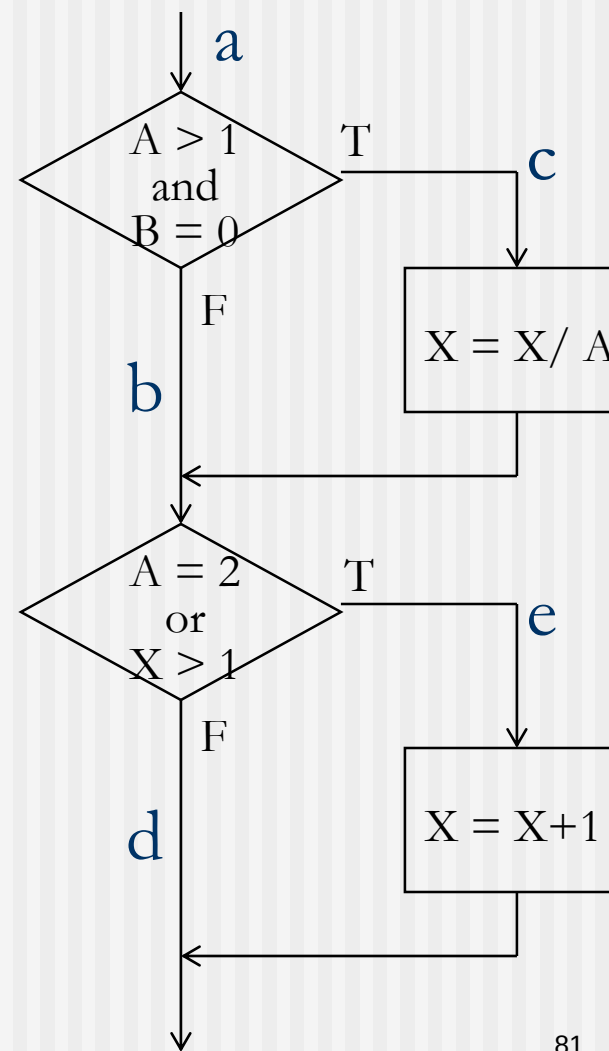
□ TCs phải bao phủ các điều kiện

- | | |
|-------------------------|-------------------------|
| 1) $A > 1, B = 0$ | 5) $A = 2, X > 1$ |
| 2) $A > 1, B \neq 0$ | 6) $A = 2, X \leq 1$ |
| 3) $A \leq 1, B = 0$ | 7) $A \neq 2, X > 1$ |
| 4) $A \leq 1, B \neq 0$ | 8) $A \neq 2, X \leq 1$ |

□ Test cases:

- | | |
|--------------------------|---------------|
| 1) $A = 2, B = 0, X = 4$ | (bao phủ 1,5) |
| 2) $A = 2, B = 1, X = 1$ | (bao phủ 2,6) |
| 3) $A = 1, B = 0, X = 2$ | (bao phủ 3,7) |
| 4) $A = 1, B = 1, X = 1$ | (bao phủ 4,8) |

Kiểm thử phần mềm



IV.5 Kỹ thuật kiểm thử dựa trên kinh nghiệm

- ❑ Error guessing
- ❑ Exploratory testing

Đoán lỗi(Error guessing)

- ❑ Được sử dụng sau khi áp dụng các kĩ thuật hình thức khác
- ❑ Có thể tìm ra 1 số lỗi mà các kĩ thuật khác có thể bỏ lỡ
- ❑ Không có qui tắc chung

Kiểm thử thăm dò (Exploratory testing)

- ❑ Hoạt động thiết kế test và thực thi test được thực hiện song song
- ❑ Các chú ý sẽ được ghi chép lại để báo cáo sau này
- ❑ Khía cạnh chủ chốt: learning(cách sử dụng, điểm mạnh, điểm yếu của PM)

IV.6 Chọn kĩ thuật kiểm thử

- ❑ Mỗi kĩ thuật riêng lẻ chỉ hiệu quả với 1 nhóm lỗi cụ thể
- ❑ Các yếu tố ảnh hưởng đến việc chọn kĩ thuật kiểm thử:
 - Loại hệ thống
 - Tiêu chuẩn quy định
 - Yêu cầu khách hàng hoặc hợp đồng
 - Mức độ rủi ro, loại rủi ro
 - Mục tiêu kiểm thử
 - Tài liệu có sẵn

Chọn kĩ thuật kiểm thử

- Kiến thức của testers
- Thời gian và ngân sách
- Mô hình phát triển PM
- Kinh nghiệm về các loại lỗi đã được tìm ra trước đó